

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 孙广润, 刘钰, 蔡合森 箱子编号 18 实验项目编号 实践任务 17,18,19
实验名称 存储管理单元设计

一、 TLB 模块设计

TLB 模块内部的主体应是一个二维组织结构的查找表。查找表的每一项分为两个部分, 实现三个功能读写和查找。

• 读写功能设计。

读写部分的实现较为简单, 和 regfile 实现较为相似, 只需要注意对于页的转换

。

```
1 //写操作
2 always @(posedge clk) begin
3     if(we) begin
4         tlb_e [w_index] <=w_e;
5         tlb_ps4MB[w_index] <=(w_ps ==`LARGE_PAGE_SIZE);
6
7         tlb_vppn [w_index] <=w_vppn;
8         tlb_asid [w_index] <=w_asid;
9         tlb_g [w_index] <=w_g;
10
11         tlb_ppn0 [w_index] <=w_ppn0;
12         tlb_plv0 [w_index] <=w_plv0;
13         tlb_mat0 [w_index] <=w_mat0;
14         tlb_d0 [w_index] <=w_d0;
15         tlb_v0 [w_index] <=w_v0;
16
17         tlb_ppn1 [w_index] <=w_ppn1;
18         tlb_plv1 [w_index] <=w_plv1;
19         tlb_mat1 [w_index] <=w_mat1;
20         tlb_d1 [w_index] <=w_d1;
21         tlb_v1 [w_index] <=w_v1;
22     end
23     else if(!we && invtlb_valid) begin
24         tlb_e <=~invtlb_op_num[invtlb_op]; //对于 INVTLB 指令来说, 将对应 TLB 表项无效的操作就是将 inv_match[i] 等于 1 对应的 tlb_e[i] 置为
25         0。
26     end
27 end
28 //读操作
29 assign r_e = tlb_e [r_index];
30 assign r_vppn =tlb_vppn [r_index];
31 assign r_ps = tlb_ps4MB[r_index] ?`LARGE_PAGE_SIZE :`SMALL_PAGE_SIZE;
32 assign r_asid =tlb_asid [r_index];
33 assign r_g = tlb_g [r_index];
34
35 assign r_ppn0 =tlb_ppn0 [r_index];
36 assign r_plv0 =tlb_plv0 [r_index];
37 assign r_mat0 =tlb_mat0 [r_index];
38 assign r_d0 = tlb_d0 [r_index];
39 assign r_v0 = tlb_v0 [r_index];
40
41 assign r_ppn1 =tlb_ppn1 [r_index];
42 assign r_plv1 =tlb_plv1 [r_index];
43 assign r_mat1 =tlb_mat1 [r_index];
44 assign r_d1 = tlb_d1 [r_index];
45 assign r_v1 = tlb_v1 [r_index];
```

● 查找功能设计。

并行比较产生 16 位的 Match 信号,生成之后在依据哪一位 match 的值为 1 进行选择,最后使用这 16 位的 match 信号作为查找信号的结果,match0 不为全 0 说明 0 号通道查找到了匹配的页表项,match1 不为全 0 说明 1 号通道查找到了匹配的页表项。match0 或 match1 中哪一位为 1 则说明命中了对应位的页表项。然后再根据输入的 s_va_bit12 或者 s_vppn 决定使用匹配到的 TLB 表项的哪一页的查找结果。

```
1  genvar i;
2  generate
3      for(i =0; i < TLBNUM; i =i +1)begin
4          assign match0[i] =(s0_vppn[18:10]==tlb_vppn[i][18:10])
5              && (tlb_ps4MB[i] ||s0_vppn[9:0]==tlb_vppn[i][9:0])
6              && ((s0_asid==tlb_asid[i]) ||tlb_g[i])
7              && tlb_e[i];
8          assign match1[i] =(s1_vppn[18:10]==tlb_vppn[i][18:10])
9              && (tlb_ps4MB[i] ||s1_vppn[9:0]==tlb_vppn[i][9:0])
10             && ((s1_asid==tlb_asid[i]) ||tlb_g[i])
11             && tlb_e[i];
12      end
13  endgenerate
14
15  assign s0_found =(|match0);
16  assign s1_found =(|match1);
17
18  assign s0_index =match0[ 1] ?4'd1 :
19              match0[ 2] ?4'd2 :
20              match0[ 3] ?4'd3 :
21              match0[ 4] ?4'd4 :
22              match0[ 5] ?4'd5 :
23              match0[ 6] ?4'd6 :
24              match0[ 7] ?4'd7 :
25              match0[ 8] ?4'd8 :
26              match0[ 9] ?4'd9 :
27              match0[10] ?4'd10 :
28              match0[11] ?4'd11 :
29              match0[12] ?4'd12 :
30              match0[13] ?4'd13 :
31              match0[14] ?4'd14 :
32              match0[15] ?4'd15 :
33              4'd0;
34
35  assign s1_index =match1[ 1] ?4'd1 :
36              match1[ 2] ?4'd2 :
37              match1[ 3] ?4'd3 :
38              match1[ 4] ?4'd4 :
39              match1[ 5] ?4'd5 :
40              match1[ 6] ?4'd6 :
41              match1[ 7] ?4'd7 :
42              match1[ 8] ?4'd8 :
43              match1[ 9] ?4'd9 :
44              match1[10] ?4'd10 :
45              match1[11] ?4'd11 :
46              match1[12] ?4'd12 :
47              match1[13] ?4'd13 :
48              match1[14] ?4'd14 :
49              match1[15] ?4'd15 :
50              4'd0;
```

将 TLB 各项对应信息填入查找端口

```
1  assign page_0      = tlb_ps4MB[s0_index] ?s0_vppn[8] :s0_va_bit12;
2  assign s0_ps       = tlb_ps4MB[s0_index] ?`LARGE_PAGE_SIZE :`SMALL_PAGE_SIZE;
3  assign s0_ppn      = page_0 ?tlb_ppn1[s0_index] :tlb_ppn0[s0_index];
4  assign s0_plv      = page_0 ?tlb_plv1[s0_index] :tlb_plv0[s0_index];
5  assign s0_mat      = page_0 ?tlb_mat1[s0_index] :tlb_mat0[s0_index];
6  assign s0_d        = page_0 ?tlb_d1 [s0_index] :tlb_d0 [s0_index];
```

```

7 assign s0_v      = page_0 ?tlb_v1 [s0_index] :tlb_v0 [s0_index];
8
9
10 assign page_1    = tlb_ps4MB[s1_index] ?s1_vppn[8] :s1_va_bit12;
11 assign s1_ps     = tlb_ps4MB[s1_index] ?`LARGE_PAGE_SIZE` :`SMALL_PAGE_SIZE`;
12 assign s1_ppn    = page_1 ?tlb_ppn1[s1_index] :tlb_ppn0[s1_index];
13 assign s1_plv    = page_1 ?tlb_plv1[s1_index] :tlb_plv0[s1_index];
14 assign s1_mat    = page_1 ?tlb_mat1[s1_index] :tlb_mat0[s1_index];
15 assign s1_d      = page_1 ?tlb_d1 [s1_index] :tlb_d0 [s1_index];
16 assign s1_v      = page_1 ?tlb_v1 [s1_index] :tlb_v0 [s1_index];

```

• INVTLB 指令。

依据书中所讲, 将各操作的匹配分解成若干“子匹配”的逻辑组合, 组织成四个子匹配条件, 然后利用这四个子条件继续组合进行匹配。对 op_num 从 1 到 6 分别进行对应的匹配, 每个值都有对应的匹配条件, 这些匹配条件可以利用上述的四个子条件进行得到。最后对于 INVTLB 指令来说, 将对应 TLB 表项无效的操作就是将 inv_match[i] 等于 1 对应的 tlb_e[i] 置为 0。

```

1 generate
2   for(i =0; i <TLBNUM; i =i +1) begin
3     assign cond1[i] =~tlb_g[i];
4     assign cond2[i] =tlb_g[i];
5     assign cond3[i] =(s1_asid ==tlb_asid[i]);
6     assign cond4[i] =(s1_vppn[18:10] ==tlb_vppn[i][18:10])
7       && (tlb_ps4MB[i] || (s1_vppn[9:0] ==tlb_vppn[i][9:0]));
8   end
9 endgenerate
10
11 assign invtlb_op_num[0] =cond1|cond2;
12 assign invtlb_op_num[1] =cond1|cond2;
13 assign invtlb_op_num[2] =cond2;
14 assign invtlb_op_num[3] =cond1;
15 assign invtlb_op_num[4] =cond1 &cond3;
16 assign invtlb_op_num[5] =cond1 &cond3 &cond4;
17 assign invtlb_op_num[6] =(cond2 |cond3) &cond4;

```

二、 添加 TLB 相关指令和 CSR 寄存器

这个实验主要是将上个实验写的 TLB 集成到我们写的 CPU 中, 并且实现 TLB 的相关指令, 在 csr 模块添加对应寄存器。

• 增加 TLBSRCH、TLBRD、TLBWR、TLBFILL、INVTLB 指令。。

在本实验新增的五条指令中, TLBSRCH、TLBRD、TLBWR、TLBFILL 都会在 TLB 和 CSR 之间进行数据的传递, INVTLB 并不负责在 TLB 和 CSR 之间传递数据, 它的数据来源也并非 csr 寄存器, 而是通用寄存器。并且会按照 OP_NUM 对 TLB 进行无效化处理。

对于这些信号的具体实现, 对应的信号和有效信号 (有效是对应信号为一与上 ID——vaild) 直接译码, 然后添加或者复用对应的数据通路。然后对原先的部分更新部分如下:

```

1
2 assign gr_we      = ~inst_st_b &~inst_st_h &~inst_st_w &~inst_beq &~inst_bne &~inst_b &~inst_blt &~inst_bge &~inst_bltu &~
   inst_bgeu &~inst_ertn &~inst_syscall &~inst_INVTLB &~inst_TLBFILL &~inst_TLBRD &~inst_TLBWR &~inst_TLBSRCH;
3
4
5 assign exc_ine    = ~(inst_add_w |inst_sub_w |inst_slt |inst_sltu |inst_nor |i
6   | inst_blt |inst_bge |inst_bltu |inst_bgeu
7   | inst_csrrd |inst_csrwr |inst_csrchg
8   | inst_ertn |inst_syscall |inst_break
9   | inst_rdcntvl_w |inst_rdcntvh_w |inst_rdcntid_w) &&ID_valid;
10
11 + | inst_rdcntvl_w |inst_rdcntvh_w |inst_rdcntid_w
12 + | inst_TLBFILL |inst_TLBSRCH |inst_TLBRD |inst_TLBWR |inst_INVTLB_op) &&ID_valid;

```

```

12
13
14 assign csr_rvalue = csr_num == `CSR_CRMD ? csr_crmd_rvalue :
15                 csr_num == `CSR_TCFG ? csr_tcfg_rvalue :
16                 csr_num == `CSR_TVAL ? csr_tval_rvalue :
17                 csr_num == `CSR_TICLR ? csr_ticlr_rvalue :
18 +                 csr_num == `CSR_TLBEHI ? csr_tlbehi_rvalue :
19 +                 csr_num == `CSR_TLBELO0 ? csr_tlbelo0_rvalue :
20 +                 csr_num == `CSR_TLBELO1 ? csr_tlbelo1_rvalue :
21 +                 csr_num == `CSR_TLBIDX ? csr_tlbidx_rvalue :
22 +                 csr_num == `CSR_ASID ? csr_asid_rvalue :
23 +                 csr_num == `CSR_DMW0 ? csr_dmw0_rvalue :
24 +                 csr_num == `CSR_DMW1 ? csr_dmw1_rvalue :
25 +                 csr_num == `CSR_TLBRENTY ? csr_tlbrentry_rvalue :
26                 32'b0;

```

关于 CSR 的具体实现,大体都是一样的。复位时清零,在写 CSR 按照对应条件更新,执行对应命令从 TLB 更新。这里仅举一例:

```

1 //ASID 的 ASID 域
2 always @(posedge clk) begin
3     if(reset) begin
4         csr_asid_asid <= 4'b0;
5     end
6     else if(csr_we && csr_num == `CSR_ASID) begin
7         csr_asid_asid <= csr_wmask[`CSR_ASID_ASID] & csr_wvalue[`CSR_ASID_ASID]
8             | ~csr_wmask[`CSR_ASID_ASID] & csr_asid_asid;
9     end
10    else if(inst_TLBRD_valid && r_e) begin
11        csr_asid_asid <= r_asid;
12    end
13    else if(inst_TLBRD_valid && !r_e) begin
14        csr_asid_asid <= 4'b0;
15    end
16
17 end

```

三、虚实地址转换的过程和 TLB 例外

- 虚实地址转换过程。

本实验需要支持的虚实地址翻译模式有两种:直接地址翻译模式和映射地址翻译模式。控制寄存器 CRMD 中的 DA 域与 PG 域决定了采用何种地址翻译模式:当 DA 域为 1,PG 域为 0 时,MMU 处于直接地址翻译模式,在这种映射模式下,物理地址直接等于虚拟地址;当 DA 域为 1,PG 域为 1 时,MMU 处于映射地址翻译模式,如果能命中且当前特权等级在该配置窗口中被允许,则采用直接映射模式,如果两个配置窗口都不命中或特权等级不合规,则采用页表映射模式。

```

1
2 +assign wb_pc =(exc_adev || exc_tlbr_inst || exc_pif || exc_ppi_inst) ? pc :
3 +             (exc_ale || exc_tlbr_data || exc_pil || exc_pis || exc_pme || exc_ppi_data) ? pc_EX :
4 +             pc_ID;
5
6
7 assign wb_vaddr =(exc_adev || exc_tlbr_inst || exc_pif || exc_ppi_inst) ? pc :
8             (exc_ale || exc_tlbr_data || exc_pil || exc_pis || exc_pme || exc_ppi_data) ? alu_result :
9             pc_ID;
10
11
12 +assign data_sram_addr_EX = da ? alu_result :
13 +
14 +             pg ? (
15 +                 dmw_hit1 ? {addr_pseg1, alu_result[28:0]} :
16 +                 (s1_ps == 12) ? {s1_ppn[19:0], alu_result[11:0]} :

```

```

16 +             {s1_ppn[19:9], alu_result[20:0]}
17 +             ) : 32'b0;
18
19 +assign addr_vseg0 =pc[31:29];
20 +assign addr_vseg1 =alu_result[31:29];
21
22
23 +assign s0_vppn =pc[31:13];
24 +assign s0_va_bit12 =pc[12];
25 +assign s0_asid =asid;
26 +
27 +assign s1_vppn =(data_sram_req_EX &&EX_valid) ?alu_result[31:13] :
28 +             inst_TLBSRCH ?tlbehi_vppn :
29 +             inst_INVTLB ?rf_rdata2[31:13] : 32'b0;
30 +assign s1_va_bit12 =(data_sram_req_EX &&EX_valid) ?alu_result[12] :
31 +             inst_TLBSRCH ?1'b0 :
32 +             inst_INVTLB ?rf_rdata2[12] : 1'b0;
33 +assign s1_asid =(inst_INVTLB) ?rf_rdata1[9:0] :asid;
34
35
36
37 +assign data_sram_addr_EX =da ?alu_result :
38 +             pg ?(
39 +                 dmw_hit1 ?{addr_pseg1, alu_result[28:0]}:
40 +                 (s1_ps ==12)? {s1_ppn[19:0], alu_result[11:0]}:
41 +                 {s1_ppn[19:9], alu_result[20:0]}
42 +             ) : 32'b0;
43
44
45 +assign inst_sram_addr =da ?pc :
46 +             pg ?(
47 +                 dmw_hit0 ?{addr_pseg0, pc[28:0]}:
48 +                 (s0_ps ==12)? {s0_ppn[19:0], pc[11:0]}:
49 +                 {s0_ppn[19:9], pc[20:0]}
50 +             ) : 32'b0;

```

• TLB 例外处理。

MMU 相关异常都与页映射地址翻译模式下 TLB 的查找结果相关,所需的各种结果信息在实现 TLB 模块时都已经设置了相应的输出接口,然后将前面在取指和访存两部分生成的异常判断结果沿流水线逐级传递至写回级,然后在写回级更新相关的 CSR 寄存器、触发异常、清空流水线并跳转到异常入口处在处理例外逻辑时,还要注意这些例外的优先级

```

1 +assign IF_readygo =(inst_addr_rcv &&inst_sram_data_ok) ||exc_adev ||exc_tlbr_inst ||exc_pif ||exc_ppi_inst;
2 +assign ID_readygo =valid_r ?(!hit_wait &&!TLBSRCH_wait) :1'b1;//访存前递阻塞
3
4 +assign exc_tlbr_inst =da ?1'b0 :(!dmw_hit0 &&!s0_found &&IF_valid);
5 +assign exc_pif =da ?1'b0 :(!s0_v &&IF_valid);
6 +assign exc_tlbr_data =da ?1'b0 :(!dmw_hit1 &&!s1_found &&data_sram_req_EX &&EX_valid);
7 +assign exc_pil =da ?1'b0 :(!s1_v &&!data_sram_wr_EX &&data_sram_req_EX &&EX_valid);
8 +assign exc_pis =da ?1'b0 :(!s1_v &&data_sram_wr_EX &&data_sram_req_EX &&EX_valid);
9 +assign exc_pme =da ?1'b0 :(!s1_d &&data_sram_wr_EX &&data_sram_req_EX &&EX_valid);
10 +
11 +assign exc_ppi_inst =da ?1'b0 :((plv >s0_plv) &&IF_valid);
12 +assign exc_ppi_data =da ?1'b0 :((plv >s1_plv) &&data_sram_req_EX &&EX_valid);
13 +
14 +assign exc_adev =(pc[1:0] !=2'b00) &&IF_valid;
15 +assign exc_ale =(data_sram_type_tag_EX[2] &&alu_result[0] !=1'b0
16 +             || data_sram_type_tag_EX[1] &&alu_result[1:0] !=2'b0) &&EX_valid;
17
18 +assign exc_at_ID =exc_break ||exc_syscall ||exc_adev ||exc_ine ||exc_tlbr_inst ||exc_pif ||exc_ppi_inst;
19 +assign exc_at_EX =exc_ale ||exc_tlbr_data ||exc_pil ||exc_pis ||exc_pme ||exc_ppi_data;
20 +
21 +assign exc_of_addr =exc_tlbr_inst ||exc_pif ||exc_tlbr_data ||exc_pil ||exc_pis ||exc_pme ||exc_ppi_inst ||exc_ppi_data;
22
23 assign wb_ecode =has_int ? 'ECODE_INT :

```

```

24         exc_adeft ? `ECODE_ADE :
25 +         exc_tlbtr_inst? `ECODE_TLBR :
26 +         exc_pif ? `ECODE_PIF :
27 +         exc_ppi_inst? `ECODE_PPI :
28         exc_ale ? `ECODE_ALE :
29 +         exc_tlbtr_data? `ECODE_TLBR :
30 +         exc_pil ? `ECODE_PIL :
31 +         exc_pis ? `ECODE_PIS :
32 +         exc_ppi_data? `ECODE_PPI :
33 +         exc_pme ? `ECODE_PME :
34         exc_syscall ? `ECODE_SYS :
35         exc_break ? `ECODE_BRK :
36         exc_ine ? `ECODE_INE :

```

TLBSRCH 与 INVTLB 处于 ID 级,但是要与处于 EX 级的访存指令共用一个 TLB 查询端口,因此需要冲突时将 TLB 相关指令延后一周期

```

1 assign TLBSRCH_wait =(inst_TLBSRCH ||inst_INVTLB) &&ID_valid &&(data_sram_req_EX &&EX_valid);

```

DMW hit control 的设计

```

1 +assign dmw_allow0 =(csr_crmd_plv ==0) &&csr_dmw0_plv0 ||(csr_crmd_plv ==3) &&csr_dmw0_plv3;
2 +assign dmw_allow1 =(csr_crmd_plv ==0) &&csr_dmw1_plv0 ||(csr_crmd_plv ==3) &&csr_dmw1_plv3;
3 +assign dmw_hit0 =dmw_allow0 &&(addr_vseg0 ==csr_dmw0_vseg) ||dmw_allow1 &&(addr_vseg0 ==csr_dmw1_vseg);
4 +assign dmw_hit1 =dmw_allow0 &&(addr_vseg1 ==csr_dmw0_vseg) ||dmw_allow1 &&(addr_vseg1 ==csr_dmw1_vseg);
5 +assign addr_pseg0 =(addr_vseg0 ==csr_dmw0_vseg) ?csr_dmw0_pseg :
6 +         (addr_vseg0 ==csr_dmw1_vseg) ?csr_dmw1_pseg :3'b000;
7 +assign addr_pseg1 =(addr_vseg1 ==csr_dmw0_vseg) ?csr_dmw0_pseg :
8 +         (addr_vseg1 ==csr_dmw1_vseg) ?csr_dmw1_pseg :3'b000;

```

四、 实验分工

• 实践十七,实践十八,实践十九。

- * 刘钰:实践任务十八;
- * 孙广润:实践任务十九;实验报告;
- * 蔡合森:实践任务十七,实验报告;